OBSERVATIONS · DOCUMENTATION · MACHINE READERS

Sentence-Level Rules for AI-Ready Documentation

Documentation that was written for people is now being read by machines as well. The structural conventions for that situation are well-established. The sentence-level patterns that affect how reliably the text can be interpreted are a separate matter, and worth describing.

By Avalynn Circe

Documentation has had one primary audience for most of its history. The reader was a person, sitting at a keyboard or holding a printed manual, looking for an answer. Everything the field learned about writing good documentation was learned in that situation. Use second person. State the prerequisites first. Keep examples runnable. Number the steps. The conventions cohered around what worked for human readers, and they work well.

In the last few years, documentation has acquired a second audience. Retrieval systems read it. Large language models read it. Internal copilots and external support agents read it. The same pages serve both audiences at once, and the new audience reads differently from the old one. It does not skim. It does not skip adjacent content because it looks unrelated. It does not understand that the third paragraph is a parenthetical aside that the writer meant to delete. It takes what it retrieves and incorporates all of it into whatever it produces next.

Information architecture, content models, and metadata are the structural conventions that have emerged in response. They are well-discussed and increasingly well-implemented. Page scope, content type templates, deprecation flags, version tags: the field has thought hard about these and continues to. What follows is an observation about a different layer of the same situation.

What Retrieval Sees

Most production retrieval-augmented systems work by breaking documents into chunks of a few hundred tokens each, embedding each chunk as a high-dimensional vector, and returning the chunks whose vectors are closest to the vector of the query. Sometimes a sparse keyword index runs alongside the vector store. The retrieved chunks become context for a language model, which writes the eventual response.

Two facts about this process are relevant to writing. First, the chunk that gets retrieved is usually a few sentences, not a whole page. Whatever those sentences say, in isolation, is what the model sees. Second, the model treats the retrieved text as authoritative source material. It does not know which sentences were meant rhetorically, which were placeholders, which were transitional connective tissue, and which were the load-bearing claims. Every sentence is taken at face value.

Both of these facts have consequences for how individual sentences behave.

Patterns That Behave Predictably

Some sentence-level patterns are common in good human-facing writing and produce reliable behavior when retrieved. Direct declarative statements. One claim per sentence. Specific subjects and verbs. Definitions before usage. Concrete numbers and names. These patterns survive being lifted out of context because each sentence contains its own meaning, intact.

Consider a sentence like this:

```
The Authorized Duration field accepts values between 15 minutes and 8 hours.
Values outside that range return a 422 error.
```

Lifted out of any surrounding context, those two sentences still convey the same constraint they conveyed in context. A retrieval system that returned only these two sentences would give a language model a complete picture of the rule. The model could answer questions about the field, about its range, about its failure mode, without needing to see anything else.

Patterns That Behave Less Predictably

Other patterns, common in good human-facing writing for stylistic or rhetorical reasons, behave differently when retrieved. They were written assuming the reader had read the surrounding paragraph. When the surrounding paragraph is gone, the sentence reads differently than the writer intended.

Here are some of those patterns, with notes on what happens to each in retrieval.

Negation Used for Emphasis

A common rhetorical move is to set up a familiar idea, dismiss it, and substitute a more accurate one. "It is not just code, it is craft." "We do not write documentation. We design experiences." The construction works for human readers because they are present for both halves and understand the second to be the intended claim.

In retrieval, the two halves can become separated. A chunk boundary can fall between them. The first half can be retrieved without the second, and the language model takes the first half at face value. "It is not just code" becomes a standalone statement, which is not what the writer meant. Even when both halves are retrieved together, the model now has two competing claims in its context window, and the rhetorical relationship between them is implicit. The model has to infer that the second sentence overrides the first.

The pattern is harmless in human reading because human readers handle rhetorical overrides naturally. In machine reading, the override is one more inference that may or may not be made correctly.

Pronouns That Reach Backward

"This is why the field has converged on the convention." "It is the reason we use that approach." "That is the constraint we work within."

Pronouns referring to a previously stated subject work in continuous prose. In a chunk that begins mid-paragraph, the referent is gone. The sentence is technically grammatical and semantically empty. "This is why we use that approach" tells a retrieval system nothing about what "this" or "that" refers to.

The same sentence with the noun phrase restored loses nothing in human reading and gains everything in machine reading. "Real-time write-back is why the eligibility cache uses a five-minute TTL." The sentence stands alone.

Hedging Language

"This is generally the right approach." "You should usually start with the eligibility check." "In most cases, the system returns a 200."

Hedging language in human-facing prose communicates appropriate epistemic humility. The writer is acknowledging that edge cases exist. The reader understands that the qualifier scopes the claim and continues reading.

In retrieval, the qualifier and the claim travel together but the boundary case the qualifier was guarding against is often elsewhere in the document. The language model receives "generally," "usually," "in most cases" without the conditions that would make those qualifiers concrete. It now has to choose between treating the claim as true (ignoring the qualifier) or treating it as uncertain (which prevents it from giving the reader a useful answer).

Removing the hedges and stating the actual condition produces a more useful retrieval target. "The system returns a 200 when the request body validates against the schema. Validation failures return a 422." Both branches are now present in the chunk, and neither branch is hedged.

Stylistic Triples

Three-part lists used for rhythm are a long tradition in English prose. They sound complete. They feel definitive. They are also frequently constructed for rhythm rather than enumeration, which means the third item is often a softer or vaguer version of the first two.

"The system is fast, scalable, and reliable." "The writer is precise, attentive, and thoughtful." "The architecture is modular, extensible, and clean."

A human reader processes these as gestures of completeness. A retrieval system processes each item as a separate claim and a language model may treat each item as something it can elaborate on. If "reliable" was the rhetorical filler in the triple, the model may still produce a paragraph about reliability that is not supported by the rest of the document.

Genuine enumerations behave differently. "The system supports PostgreSQL, MySQL, and SQLite" enumerates three specific items, each of which the model can correctly answer questions about. The pattern looks the same on the surface but the underlying content is concrete.

Scene-Painted Claims

Some sentences communicate competence or quality through scenes rather than evidence. "Equally at home writing the docs and reading the source." "Built to handle whatever production throws at it." "The kind of platform that engineering teams trust."

In human-facing copy, these sentences function as emotional cues. The reader recognizes the genre and adjusts expectations. In machine retrieval, the sentences are taken as factual claims. "Built to handle whatever production throws at it" becomes a property of the platform, retrievable in answer to questions about platform capabilities, supported by no specific evidence and disconfirmable by nothing in the surrounding text.

Replacing scene-painted claims with the specific evidence they were gesturing at improves machine behavior and tends to improve human behavior too. "Handles 12,000

requests per second on the reference benchmark" is something a model can correctly summarize, compare, and cite.

What These Patterns Have in Common

Every pattern described above shares a structural property. Each one works for human readers because humans read with context: the surrounding paragraph, the structure of the page, the writer's voice and intent, the genre conventions of the document. When the surrounding context is removed, the sentence loses information it had been relying on the context to supply.

Machine retrieval removes the surrounding context as a routine matter. The chunk that gets retrieved is what the language model sees. The rest of the page might as well not exist for the duration of that response.

This is not a flaw in machine retrieval. It is a property of how the retrieval works, and the same property is what makes retrieval fast and scalable to documents of arbitrary size. The property has implications for writing because writing developed in a context where it never had to assume.

> *Writing that survives being lifted out of context tends to also be writing that humans read more easily. The two audiences are not in tension.*

Sentence-Level Conventions for the Situation

A short list of conventions that follow from the observations above. None of these are rules in any binding sense; they are working notes from one writer attempting to produce documentation that behaves consistently when read by both audiences.

Make every sentence stand alone. Read each sentence as if it were the only one a reader would see. If the sentence depends on the previous sentence for its meaning, restate the dependency.

Replace pronouns with their referents when the referent is more than a sentence away. "The cache" instead of "it" if "the cache" was last named two paragraphs ago.

State conditions instead of hedging certainty. "When the input validates" instead of "usually." "For requests over 1MB" instead of "in most cases." The qualifier is a placeholder for the condition; the condition is what the reader needs.

Use enumerations only when the items are independently meaningful. Three items because there are three things, not because three sounds rhythmically complete.

Replace rhetorical contrasts with direct claims. The positive half of "not X, but Y" is usually the only half that needed to be said. State that half.

Specify what was being gestured at. If a sentence praises a quality, name the evidence for the quality in the same sentence, or replace the praise with the evidence.

On Naming Conventions and Codifying Them

Working through these patterns produced a longer list than is reasonable to keep in working memory. The list became a written specification, which has gone through several drafts and continues to evolve. The current version is called the Circe Editorial Standard for Gen AI Content. It carries an explicit work-in-progress label because the patterns are still being identified and the rules are still being refined.

The standard does not assert that text written without it is bad. It does not assert that documentation produced by any particular workflow is broken. It exists because keeping a written record of editorial decisions is easier than re-deriving them every time a document is reviewed. It is a working tool, not a position.

Anyone curious about its current contents is welcome to read them. Anyone with patterns to add is welcome to suggest them. The standard is a draft, and a draft is the form of a thing that benefits most from being shared.

Avalynn Circe is a software developer, business analyst, and technical writer based in St. Paul, MN. She is the author of JavaScript: The Parts, a 350-page technical reference for working JavaScript developers, available at leanpub.com/jsparts.